phase_2_Azra

Azra Bayrak

February 2026

0.1 Technical Foundations: The Shared Kernel

In order to guarantee the integrity of the game engine and meet the requirements of the authoritative server model, the development of the business logic relies on several strict concepts of software architecture and Object-Oriented Programming (OOP) in Java.

- **Type Safety (Enumerations):** Fundamental attributes such as suits (`Suit`) and ranks (`Rank`) are modeled using `enum`. This prevents the instantiation of illegal values (e.g., a "15 of Spades") and secures the deserialization of network packets.
- **Entity Immutability (Java `records`):** In a critical transactional environment, the state of a dealt card must never be alterable. The `Card` class is therefore implemented as a `record` (an immutable data structure without write accessors), thus eliminating any risk of memory corruption.
- **Algorithmic Sorting (`Comparable`):** To meet the performance constraints of the hand evaluator (`HandEvaluator`) during the *Showdown*, the `Card` object implements the `Comparable<Card>` interface. This mathematically defines the value hierarchy in order to optimize list sorting in constant time.
- **Cryptographic Random Generation (CSPRNG):** In accordance with gambling standards, shuffling the `Deck` prohibits the use of the standard pseudo-random generator (`Math.random()`). The system relies exclusively on `java.security.SecureRandom`, ensuring that card distribution is mathematically unpredictable and resistant to inference attacks.
- **Distributed Traceability (`UUID`):** Identification within the `Player` class is handled via UUIDs (Universally Unique Identifier). This ensures collision-free interoperability with the primary key (`id_joueur`) of the `T_JOUEURS` table in the PostgreSQL database.

1 Phase 1

1.1 Packages (package com.quantum.model;

What is it? A package in Java is like a folder on your computer. Why do we do this? In your project, you have many files. If we put the graphical interface (JavaFX), networking (Sockets), and game logic in the same folder, it becomes a nightmare. The package model means "Data Model". We only put the "pure" game objects there (the mathematical entities) that have no idea how they are drawn on the screen. This is what we called the "Anemic POJOs".

1.2 Enumerations (enum Suit, enum Rank)

What is it? An enum (enumeration) is a strict list of possible choices. Why do we do this? Imagine we use text (String) for the suit of a card. Someone could create a card "Heart", another "Hearts" (with an 's'), and a hacker could send the suit "Banana". The program would crash. By creating an enum Suit HEARTS, DIAMONDS, CLUBS, SPADES , we force Java to accept only these 4 exact values. It is absolute anti-bug security. For the rank (Rank), we associate a numeric value (e.g., Jack equals 11) so that the computer can easily compare which card is stronger.

1.3 The record keyword (Card immutability)

What is it? Recently introduced in Java, a record is a fast way to create a "read-only" class. It has no "Setters" (you cannot do card.setRank()). Why do we do this? This is the concept of immutability. In a gambling game, once a card is dealt (e.g., the Ace of Spades), it must never be modifiable. If we used a normal class, a bug (or a cheater) could turn their 2 of Clubs into an Ace of Spades during the game. With a record, it is technically impossible for the program to modify it.

1.4 The `Comparable <Card>` interface

What is it? It is a "contract" given to the Card class. It forces you to write a method that explains to Java how to rank two cards against each other. Why do we do this? Later, in the HandEvaluator algorithm, we will need to find the best 5-card hand out of 7 cards. To do this quickly, we must first sort the cards from strongest to weakest. Since Java does not know whether a "King" is stronger than a "2", we define the rule here.

1.5 The Deck class and the Stack (`Stack<Card>`)

What is it? A Stack is a data structure shaped like a "Pile" (like a stack of plates). The principle is LIFO (Last In, First Out = The last placed is the first removed). Why do we do this? It is exactly like a real deck of cards: we stack the 52 cards, and when we deal (deal()), we always take the one on top.

1.6 SecureRandom (The anti-cheat shield)

What is it? It is a military-grade (cryptographic) random number generator. Why do we do this? Normally, beginners use Math.random() or java.util.Random. The problem is that these generators are simple mathematical formulas. A hacker who observes a few dealt cards can calculate the formula and predict the rest of the deck. The specifications explicitly require the use of a CSPRNG (Cryptographically Secure Pseudo-Random Number Generator) on the server to guarantee fairness. SecureRandom is the Java class that meets this requirement.

1.7 The UUID in the Player class

What is it? A UUID (Universally Unique Identifier) is a very long string unique worldwide (e.g., 123e4567-e89b-12d3-a456-426614174000). Why do we do this? Rather than saying "Player 1" and "Player 2" (which would cause problems if there are 100,000 players in the database), we assign a UUID. This is what is defined in the database (T_JOUEURS uses a UUID as the primary key). It allows a player to be identified over the network without ever making a mistake.

2 Step 2: The Standard Poker Engine.

This step is divided into two large parts: the HandEvaluator (who has the best hand at Showdown?) and the PotManager (who wins which chips, especially if there are different All-Ins?).

2.1 The HandEvaluator (Who is the winner?)

2.1.1 What is the problem?

In Texas Hold'em, each player receives 2 private cards, and there are 5 community cards on the table. That makes 7 cards total. The goal of the algorithm is to extract the best 5-card combination from these 7 cards.

2.1.2 How does the computer read the cards?

A human immediately sees whether they have a "Straight" or a "Flush." The computer, however, is blind. If we make it run loops (for... if... else) everywhere, the server will be very slow, which is forbidden because you must resolve ties in less than 2ms.

The trick is the Histogram and Bitmasking technique:

- The program will perform a quick "count" of Ranks (how many Aces, Kings, etc.) and Suits (how many Spades, Hearts, etc.).
- If it sees the number "4" in the Rank column, Bingo! It knows there is a Four of a Kind.
- If it sees the number "5" (or more) in the Suit column, Bingo! There is a Flush.

2.1.3 The golden rule: Kickers

Imagine Alice has "Ace - King" and Bob has "Ace - Queen." On the table, there is an Ace. Both have a Pair of Aces. Who wins? The answer is Alice, thanks to her "Kicker" (the King is stronger than the Queen).

That is why the specifications require creating a HandRank object. This object will store two things:

- The Category (e.g., PAIR, FLUSH, STRAIGHT_FLUSH).
- A List of Kickers (the ranks of the remaining cards, sorted from strongest to weakest).

Thanks to this, when the server compares Alice and Bob, it first compares the categories (Pair vs Pair = Tie), then compares the list of Kickers (King vs Queen = Alice wins).

2.2 The PotManager (Managing money and Side Pots)

This is the financial part. If everyone had the same number of chips, it would be easy. But poker is not like that.

2.2.1 The nightmare of asymmetric "All-In"

Imagine this situation:

- Charlie has 100 chips.
- Bob has 500 chips.
- Alice has 1000 chips.

Charlie decides to go All-In with his 100 chips. Bob calls with 100 chips, and Alice does the same. The pot contains 300 chips. But Alice and Bob want to keep betting against each other! Bob goes All-In with his remaining 400, and Alice calls.

2.2.2 2. How the computer handles this (Algorithm B from the specifications)

The document describes the exact rule to divide the money without ever making a mistake or losing a single cent. Here is how the algorithm proceeds step by step:

- Step 1 (Sorting): The server looks at the list of all bets made by active players and takes the smallest amount. Here, it is Charlie (100).
- Step 2 (The Main Pot): It takes 100 chips from Alice, 100 from Bob, and 100 from Charlie. It creates a SidePot (the main pot) of 300 chips. Everyone is eligible to win this pot.
- Step 3 (Side Pot 1): Charlie is "empty," so we ignore him. The remaining bets are Bob (400) and Alice (400). The smallest bet is 400. It takes 400 from Bob and 400 from Alice. It creates a second SidePot of 800 chips. Only Alice and Bob are eligible to win it! (Charlie is not allowed to touch it, even if he has the best hand at the end).
- Step 4 (Distribution): At Showdown, the server first looks at the pot of 800. It compares Alice's and Bob's hands and gives the 800 to the winner. Then it looks at the pot of 300. It compares Alice, Bob, AND Charlie, and gives the 300 to the winner.

2.2.3 Summary

For the game engine to work, we must implement:

- A HandEvaluator class that reads the cards and returns a HandRank.

- A PotManager class that takes players' bets and creates separate pots when there are All-Ins.